



Adaptive parallel Louvain community detection on a multicore platform



Mahmood Fazlali^{a,*}, Ehsan Moradi^b, Hadi Tabatabaee Malazi^c

^a Department of Computer Science, Shahid Beheshti University, GC, Tehran, Iran

^b Department of Computer Engineering, Faculty of Engineering, Kermanshah Branch, Islamic Azad University, Kermanshah, Iran

^c Faculty of Computer Science and Engineering, Shahid Beheshti University, GC, Tehran, Iran

ARTICLE INFO

Article history:

Received 3 October 2016

Revised 9 July 2017

Accepted 1 August 2017

Available online 23 August 2017

Keywords:

Thread load balancing
Thread-level parallelization
Task decomposition
Multicore systems
Social networks
Nested parallelism

ABSTRACT

Community detection is a demanded technique in analyzing complex and massive graph-based networks. The quality of the detected communities in an acceptable time is an important aspect of an algorithm, which aims at passing through an ultra large scale graph, for instance a social network graph. In this paper, an efficient method is proposed to tackle Louvain community detection problem on multicore systems in the line of thread-level parallelization. The main contribution of this article is to present an adaptive parallel thread assignment for the calculation of adding qualified neighbor nodes to the community. This leads to obtain a better load balancing method for the execution of threads. The proposed method is evaluated on an AMD system with 64 cores, and can reduce the execution time by 50% in comparison with the previous fastest parallel algorithms. Moreover, it was observed in the course of the experiments that our method could find comparably qualified communities.

© 2017 Elsevier B.V. All rights reserved.

1. Introduction

A challenge in parallel processing is to determine the granularity of task decomposition in a way to achieve the best performance on multicore platforms [1]. Therefore, researchers tried to provide various decomposition methods to reach the best parallelization. However, some of the solutions for the classic problems cannot reach the best decomposition granularity anymore. Increasing the number of cores on a chip can provide an infrastructure for researchers to come to new thread-level parallelism in the decomposition. While this set-up has a significant effect in parallelization, it causes a new question to be raised. Which granularity of thread decomposition can achieve the best performance on new multicore systems? The performance can be viewed from different perspectives, including communication and synchronization overheads.

Nowadays, various fields in the science are dealing with the analysis of large graphs, especially community detection. The behavior of the community (dense sub-graphs) can be considered as a good sample of the behavior of its comprising nodes. Therefore, determining the behavior of communities may lead to the analysis of the graph behaviour. Ranking problem in search engines [2], finding effective groups in protein networks [3], user categorizing in social networks [4], and providing offers in recommendation systems [5] are practical examples of this sort of research. While the graph scale makes the problem more challenging, the current configuration of existing graphs, e.g. Facebook, which is experiencing more than 13.5 billion nodes forces researchers to think about parallel solutions. Indeed, detecting communities in such large graphs using sequential algorithms is impractical and utilizing compatible parallel solution on parallel platforms (multicore system) is an option in tackling the problem.

Among the proposed methods for addressing the community detection problem, Louvain method [6] is the most prosperous solution, which uses modularity as a qualification parameter. This sequential algorithm requires a powerful machine for analyzing massive graphs, which is usually expensive and not affordable. Therefore, researchers have conducted alternatives to accelerate this solution using parallel platforms [7–15]. As the calculation of modularity parameter in the graph is computationally intensive, a trade-off between the speedup and the community quality was considered in [8,10,14,16]. In some works [11,12], there could still be a potential to accelerate community detection by increasing the number of cores on multicore architectures. The weakness of these methods is that they do not adapt the granularity of threads by the number of available cores in the system. Finding the number of threads to calculate the modularity of added neighbor node to the community depends on the number of neighbors in a graph as well as the number of cores in the system. Therefore, choosing

* Corresponding author.

E-mail addresses: fazlali@sbu.ac.ir (M. Fazlali), ehsan@ehmo.ir (E. Moradi), h_tabatabaee@sbu.ac.ir (H. Tabatabaee Malazi).

the granularity of threads can be obtained adaptively at run-time based on this information to get the maximal acceleration in the parallelization of Louvain algorithm

In this paper, a new parallel algorithm, namely Adaptive Parallel Louvain Method (APLM), is designed to overcome the weakness mentioned above by using a hierarchical clustering method. Initially, the algorithm considers any nodes in the input graph as a community. Then, it randomly selects a community, and adaptively allocates threads to its neighbors by considering the number of free cores on the system. The threads calculate the modularity considering the neighbor to be added to the community. Then, the algorithm is able to make a decision about joining the selected nodes to the community or not. The process continues until all the nodes are visited.

There is a trade-off between achievement and overhead in the parallelization. This study can also overcome this challenging trade-off as another contribution. Decomposition granularity is achieved by dividing the calculation of the modularity parameter in joining a neighbor node among threads in an adaptive manner. This solution helps to reach a better load balancing for threads in comparison to previous algorithms. To evaluate our algorithm three levels of scaling (small, medium, and large), in social network graphs are used. The results demonstrate that the proposed algorithm outperforms the previous methods [16,17] by reducing the execution times by up to 50%. Besides, the qualification parameter is comparable to the previous best work.

The rest of the paper is organized as follows: Section 2 is dedicated to reviewing parallel community detection algorithms. In Section 3, the problem is described. Section 4 expresses the proposed algorithm in detail. Comparisons and the performance evaluation of the proposed approach are presented in Section 5. Finally, in Section 6, we draw the conclusion.

2. Literature review

With respect to the broad applications of community detection in the area of complex network analysis, different definitions were presented for the community among which two descriptions get popular by data scientists. (i) A community is defined as a set of nodes whose properties are more similar [18]. (ii) A community is a set of nodes which has more edges in a cluster rather than the other nodes in the rest of graph [19]. In other words, a community is considered as a set of graph nodes that its internal edges are denser than the external ones. In this paper, targeting the second definition, the wide-ranging literature on community detection is firstly surveyed. Then, the restricted literature on parallel Louvain community detection is discussed in Section 2.4.

2.1. Input-Output graphs

The input graphs for community detection algorithms can be grouped into two categories. The first one is ordinary graphs, comprised of homogeneous nodes and edges. In the second one, which is called a bipartite graph, the nodes are comprised of two independent and disjoint sets in which those nodes that belong to the first set can only connect to the second set nodes, and vice versa. The definition can be extended to multi-partite graphs.

The output of the community detection algorithm can be non-intersecting, overlapping, or concept graphs. In non-intersecting graphs, each node belongs to only one community. The partitioned graph is the output of most of the community detection algorithms [20,21]. In contrast, in overlapping communities, a node may associate with several communities [22], for instance, the membership of a user in several groups in a social network. The concept graphs demonstrate the nodes with similar properties. The focus of our

work is to detect non-intersecting communities, since they need a rapid online community detection.

2.2. Quality measure

There are two school of thought for assessing the quality of the detected communities.

The first one is to use experts, and reach a consensus on an interpretation for each community. This approach requires a considerable effort, and is not practical in all the applications. The second approach is to define and use a quality measure. Modularity [19] is one of the most widely accepted metrics. It is the ratio of edges inside the community to those that connect the community to the outside world in a randomly distributed graph.

Modularity is defined in Eq. (1), where $A_{i,j}$ represents the weights of the link between i and j in the input graph. According to the equation, K_i is the sum of the weight of the links connected to node i , and C_i is the community that i belongs to. $\delta(C_i, C_j)$ is the function that returns one, if two nodes i, j belong to the same community; else, it returns zero. Finally, $m = \frac{\sum_{i,j} A_{i,j}}{2}$ is the sum of weights for the links.

$$Q = \frac{1}{2m} \sum_{i,j} [A_{i,j} - \frac{k_i k_j}{2m}] \delta(C_i, C_j) \quad (1)$$

Although in some references it is mentioned that the limitation of the modularity is the capability of detecting small communities [7,16], in this paper, it is utilized to compare the quality of communities.

Weighted Community Clustering (WCC) is another quality measure introduced in [7], which considers the distribution of triangles in each sub-graph (community) in the network. The link centrality is used as a classification factor to either join a node to the community, or disjoins it from the community [23]. A different quality parameter is presented in [24] that enhances coreness centrality by considering both local and global information of the network.

2.3. Analytical approaches

Three types of analytical approaches can be applied to detect communities [25]. The most comprehensive approach, which is usually used in social networks, is the graph theory approach. It analyzes the graph structure with graph properties and algorithms [26,27]. Spectral clustering is a method that considers a similarity function for each pair of nodes to classify the graph. In [28], the ratio of eigenvalue between the first eigenvector and other eigenvectors in the graph is employed for the classification. Hierarchical clustering is another well-known idea for the classification of nodes to make communities [29]. In this method, the nodes agglomerate in each step, and are considered a single node for the next step. The main associated weakness is to increase the algorithm execution time by increasing the number of graph nodes. This issue is addressed in this study by acquiring efficient algorithm decomposition in parallelization of hierarchical clustering. Another approach is called hyper-graphs. The main contrasting point in hyper-graphs is that an edge can connect any number of nodes [30–33]. The last approach is called concept (Galois) lattice. In this approach, communities are detected in a way that individuals share the same subset of properties [33,34].

This study lines in the first category and parallel Louvain methods which have been proved to be the most efficient (considering accuracy and execution time) algorithms in this category [11,12]. Therefore, we briefly review previous versions of parallel Louvain community detection algorithms.

2.4. Parallel Louvain community detection algorithms

Blondel et al. [6] devised a hierarchical algorithm named Louvain to detect small communities based on the modularity measure. The algorithm at first considers all nodes as small communities. Then it tries to merge the communities (nodes) in order to increase modularity measure. Louvain has been a basic idea for so many community detection algorithms up to now. So, several parallel methods introduced as a parallel version of this work. In most of the previous work which employ multicore platforms, Pthread or Open-MP library are widely used to implement parallel version of Louvain. For example researchers in [35] try to parallelize the loop of calculating modularity using Open-MP library on an Opteron quad-core system with 8 GB RAM. And researchers in [8] use Open-MP library, Cray XMT, and Intel E7-8870 machines for the parallelization of Louvain. They tried to parallelize loops in modularity calculation of Louvain phases. The results show that their implementation reached a better performance on Cray XMT high-performance machine. However, the use of a specific architecture makes repeating the experiments impossible to the wide range of scholars. In the same way the authors in [12] try to parallelize Louvain, but their algorithm cannot effectively accelerate large graphs, due to the lack of an efficient decomposition strategy.

Parallel Louvain Method (PLM) is an idea proposed in [16] that consider shared-memory parallelization by using Open-MP library. It uses fine grain algorithm decomposition in calculating modularity measure of adding a neighbor node. This is one of the closest work to our research because of using the same parallel paradigm. The PLM is also extended by adding a refinement phase on every level, which yields the Parallel Louvain Method with Refinement (PLMR) algorithm; nonetheless, PLM is faster than PLMR. Authors also implement three standalone parallel algorithms: they implement a parallel version of the Label Propagation method as the PLP algorithm. In addition to these basic algorithms, they also implement a two-phase approach that combines them. It is inspired by ensemble learning in which the output of several classifiers is combined. In their case, multiple base algorithms run in parallel as an ensemble. Their solutions are then combined to form the core communities representing the consensus of all base algorithms. The graph is coarsened according to the core communities, and then assigned to a single final algorithm. Within this extensible framework, which they call the ensemble preprocessing method (EPP), they apply PLP as base algorithms and PLMR as the final algorithm. A bigger granularity for algorithm decomposition to calculate modularity was used in [17] that employs multicore platform and open-MP library. It uses a thread to calculate the modularity and obtains better speedup than previous algorithms.

As an example of efforts on GPU parallelization, researchers in [36] present the design of a parallel algorithm for community detection optimized for multi-core and GPU architectures using CUDA library. Their algorithm is based on label propagation so they have a better situation for a GPU parallelization on community detection problem. They also show that weighted label propagation can overcome typical quality issues in communities detected with label propagation. Experimental results on a benchmark named Wikipedia and also on RMat graphs using IBM Power6 microprocessor with 32 cores further, their General Purpose Graphic Processor Unit (GPGPU) based algorithm achieves 8x improvement over the Power6 performance. As another research Richard Forster in [37] shows that Louvain can be accelerated on GPU using CUDA library where the input graph matrix can be loaded in GPU memory and it has dense edges. This is not our definition especially for general social network graphs.

A trend in processor development is to gradually increase the number of cores considering shared and dynamic cache management to support the simultaneous multithreading. These technolo-

gies push programmers to use effective and adaptive thread decomposition in designing algorithms for new multi-core platforms. Therefore, finding an adaptive decomposition algorithm for Louvain has to be considered, which can handle the new generation of multicore systems with various number of cores.

In this paper, a new decomposition algorithm for community detection is proposed in which a better speedup is achieved, while the quality of the communities remains similar to the ones detected in the previous works.

3. Problem definition

Different types of communities are defined in the literature so far. The problem space that we are addressing in this paper is as follows:

The input is a graph, the abstraction of a network dataset, that is denoted as $G = (V, E)$ with a node set V of size n and an edge set E of size m . The neighbor of node A in the graph is a node B that is adjacent to A , and is connected directly by an edge to A . The graph is an undirected weighted one. The output of the method is a set of communities, which partition the node set V into disjoint subsets $\{C_1, C_2, \dots, C_k\}$. Hence, the output communities do not overlap. The solution is represented as a list of size k (number of communities), containing integer community identifiers, and node integer identifier (i.e., a mapping of nodes to communities).

The main goal is to find communities as fast as possible. That is, the running time of community detection algorithm has to be reduced. Another important parameter is the modularity, which is the negation of the fraction of the edges falls within the given groups from the expected such fraction if edges were randomly distributed. For a given division of the network's vertices into some communities, modularity reflects the concentration of edges within communities compared to a random distribution of links among all nodes regardless of modules. Fig. 1 presents the steps of creating communities from the input graph. Calculating the modularity of adding the neighbors of a node is done in parallel after Fork. This means there is not any dependency to calculate the modularities here. After forking the results, we have to do the same task for the next node. Then the new graph is created and the modularity of the graph is calculated to decide whether to start a new iteration or finish the algorithm.

4. The proposed adaptive parallel Louvain method (APLM)

Louvain method [6] is one of the accurate and fast community detection algorithms. However, as the use of modularity to satisfy the accuracy, parallelization of the algorithm on many core systems is challenging [12]. There are two alternative techniques for algorithm decomposition of parallel Louvain. The first one is to compute the gained modularity of adding a neighbor node to the community by assigning some threads in parallel. The gained modularity equation for a node contains a Sigma summation that indicates whether the neighbor node should be added to the community. This calculation is defined as Eq. (2) [6], where Σ_{in} is the sum of the weights of the links inside community, Σ_{tot} is the sum of the weights of the links incident to nodes in C , k_i is the sum of the weights of the links incident to node i , $k_{i, in}$ is the sum of the weights of the links from i to nodes in C , and m is the sum of the weights of all the links in the network.

$$\Delta Q = \left[\frac{\Sigma_{in} + k_{i, in}}{2m} - \left(\frac{\Sigma_{tot} + k_i}{2m} \right)^2 \right] - \left[\frac{\Sigma_{in}}{2m} - \left(\frac{\Sigma_{tot}}{2m} \right)^2 - \left(\frac{k_i}{2m} \right)^2 \right] \quad (2)$$

The second decomposition algorithm runs a separate thread for each neighbor node to calculate the gained modularity. It uses the

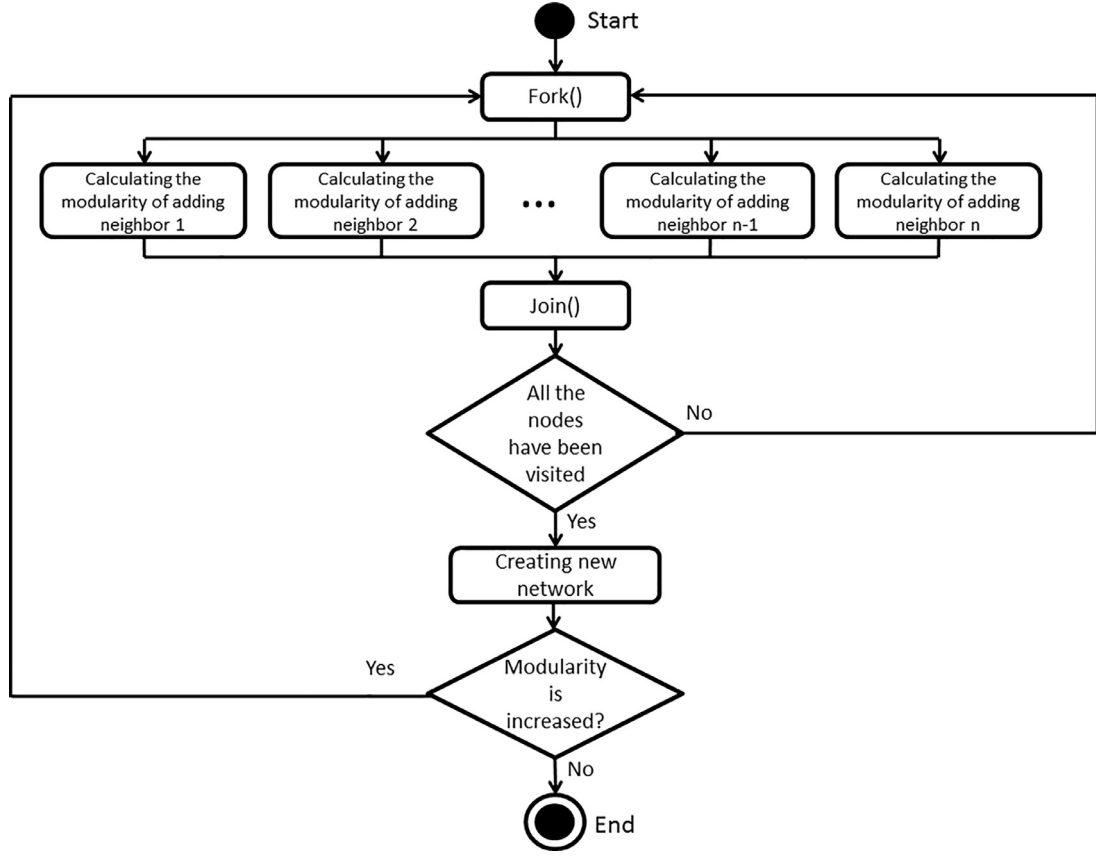


Fig. 1. The flowchart of the parallel Louvain algorithm.

coarser decomposition of grains, which can experimentally reach a better acceleration in community detection [17].

In multicore systems, the possibility of some cores remaining idle can reduce the efficiency of the community detection. Suppose that the task computes the modularity of adding a neighbor. Therefore, using fine-grain task decomposition whenever the number of tasks is less than the number of cores is the key. This is done by dividing a task and assigning each subtask to a core. In cases where the number of tasks is more than the number of cores, a coarser grain task decomposition is used. In order to assign the cores to the threads in the aforementioned cases, adaptive thread decomposition is applied, and a new parallel community detection algorithm is present, which is scalable by increasing the number of cores.

The granularity of threads is obtained depending on the number of neighbors for each node. Our input workload is a graph, and in each step of the algorithm, we should calculate the modularity of adding neighbors to the node in a parallel manner. So, data locality in our algorithm is equal to the number of neighbors. Our algorithm tries to adaptively allocate the cores to the threads to calculate the modularity based on the number of neighbors (locality of workloads).

The proposed method calculates the number of community neighbor nodes (candidate nodes), which has the potential to be added to the community. Then, it obtains the number of idle cores on the system using an Application Program Interface (API). The number of the candidate nodes and the idle cores provides the required information to assign one or more thread to calculate the modularity of the added neighbor node. In other words, at the first level of parallelism, each node received at least one core, and at the second level, the node may exploit more than one core for its local processing. Fig. 2 depicts the multilevel parallelism of mod-

ularity for the neighbors of node A. It shows a multicore system comprised of 64 idle cores. First, the method explores the neighbors of node A. To check the possibility of joining the four neighbors of node A, eight cores can be assigned for each neighbor to calculate the gain modularity Sigma. Therefore, at the first level, the method assigns cores for independent calculations of nodes, and at the second level, it provides several cores to each node for intra-node processing.

Algorithm 1 shows the pseudo code of the proposed commu-

Algorithm 1: APLM (V,E).

```

1 k=1; //Round
2 repeat
3    $N_C \leftarrow G$ ;
4   for all nodes in  $N_C$  do
5     Choose a random  $v_i$ ;
6     //OMP_DYNAMIC is disabled;
7     #threads  $\leftarrow \text{Degree}(v_i)$ ;
8     for any  $v_j \in \{(v_i, v_j) \in E\}$  do in parallel
9        $Q_j = \text{NP\_Modularity}(v_j)$ ; //Nested Parallel Modularity
10     $Q_{Max}^k \leftarrow \text{Max}(Q_j^k)$ ;
11    if  $(Q_{v_i} \geq Q_{Max}^k)$  then
12       $G = \text{new Network}()$ ;
13  k=k+1;
14 until  $(Q_{Max}^{k-1} \leq Q_{Max}^k)$ ;
  
```

nity detection method. In the first stage, all nodes are assigned to the separate communities. This is done in parallel by domain decomposition of the loop, which is depicted in line 4. Then, the al-

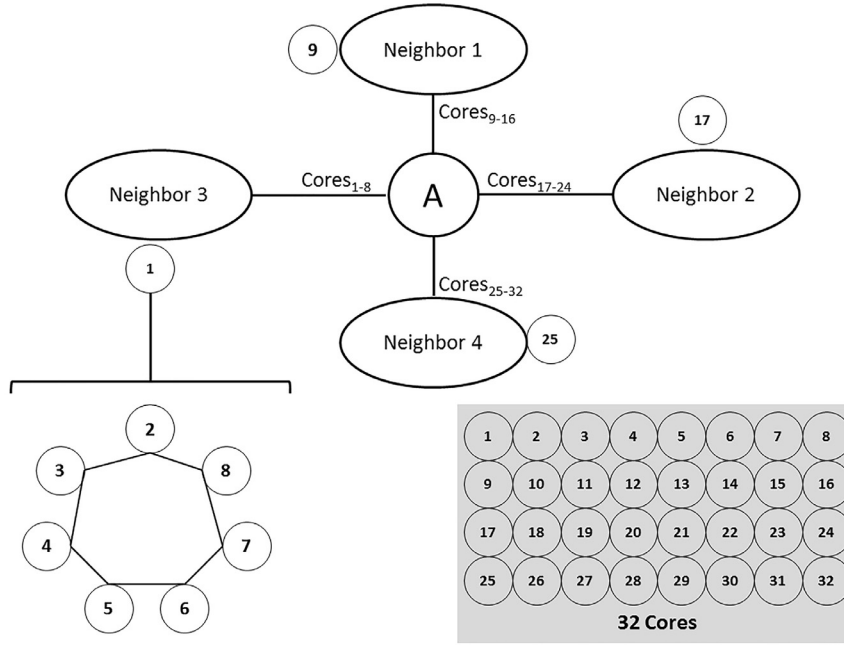


Fig. 2. Gained modularity calculation for four neighbor nodes utilizing 32 cores in APLM.

gorithm chooses a random community, and considers it as a node (line 5). Subsequently, it is the time to calculate the gained modularity of merging each neighbor node to the selected node (line 9), which can be done in parallel. The function is demonstrated in detail in Algorithm 2. Accordingly, the node will join the commu-

Algorithm 2: NP_Modularity(Node v).

```

1  $n = \text{Degree}(v)$ ; // Number of Neighbors
2  $c = \text{Number of Available Cores}$ ;
3 if  $c > n$  then
4    $t = \lceil (c - n) / n \rceil$ ; //Number of Threads
5   nested_enabled();
6   omp_set_num_threads( $t$ );
7 else
8   omp_set_num_threads(1);
9 parallel modularity calculating;
```

nity, if the calculated gained modularity increases (line 11). This process is repeated for all the neighboring nodes. Then, the algorithm starts choosing new communities, if modularity of the graph is increased, and repeats this process again. After finishing all communities, the new network is created.

The nested parallel Modularity function depicted in Algorithm 2 obtains the number of cores in the system. If the number of cores is more than the number of the neighbor nodes (line 3), it calculates the number of threads that have to be assigned for the calculation of gained modularity of adding neighbor (line 4). This is done by using nested task parallelism capability in Open-MP. It should be noted that the dynamic thread assignment of Open-MP has to be disabled to let the algorithm adaptively choose the number of threads. If the number of cores is less than the number neighbors being added, the number of threads is set to one (line 8).

To create the network in line 12 of Algorithm 1, each community is considered as a node. All links in the community are considered as a circular link, whose weight is equal to the weight of all inside links. Also, all links between the two communities are re-

placed by a link, whose weight is equal to the total weight of the replaced links. Subsequently, APLM calculates the modularity of the new graph to decide if the modularity threshold is reached or continue the community detection. This step can be implemented in parallel as well. Fig. 3 depicts three passes of merging nodes in the input graph to create communities. At first, there are several nodes (communities), and the modularity is equal to 0.8555. In each pass of the algorithm, the nodes are combined together to make larger communities. After three passes, it reaches the modularity equal to 0.8885.

Our adaptive parallel thread assignment can employ many cores on the chip to accelerate community detection. The next section will present the experimental results of utilizing APLM with various numbers of cores.

5. Experimental results

To evaluate the proposed APLM algorithm, we performed experiments using some well-known network graphs [38,39]. The implementation platform is an AMD 2.8 with 32 cores, and each core can run two threads (64 virtual cores), and the memory size is 128GB. Open-MP library is used for the parallelization, and G++ compiler version 4.8 is employed to compile C++ source code. The OS is Red Hat 4.4 distribution of Linux.

Five benchmark graphs are used to evaluate APLM performance. The specifications of them are presented in Table 1. In this table, the first column presents the name of the benchmark. The second column and the third column present the number of nodes and number of edges in the graphs. The fourth one shows the ratio of the number of edges to nodes, and the last column describes the original usage of the graph. We use various graph sizes from a small graph such as CNR-2000 to a big one (UK-2007). These graphs have various ratios for edge/node; so, the running time of the Louvain algorithms is different.

To compare the performance of the devised method, two fast previous parallel Louvain community detection approaches that have the comparable modularity quality factor to sequential Louvain are used. The first one is the Parallel Louvain Method (PLM) [16] that parallelizes the calculation of the gained modularity

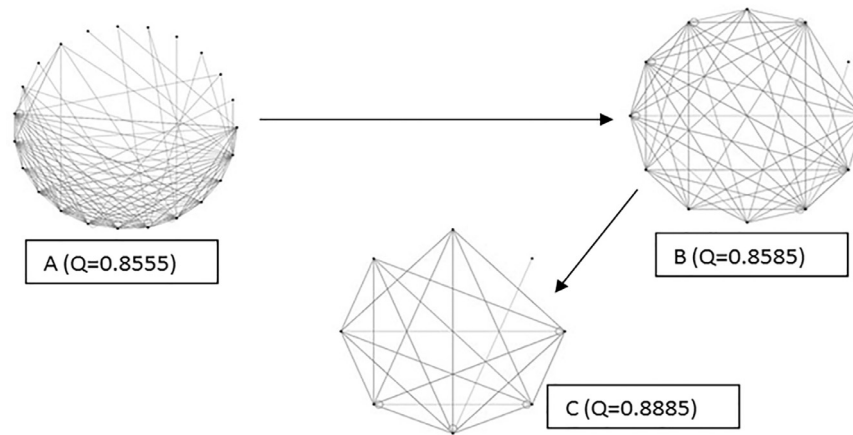


Fig. 3. The output graph in three passes of APLM algorithm.

Table 1
Specifications of the benchmark graphs.

Benchmark	# of Nodes	# of Links	$\frac{\# \text{ of Links}}{\# \text{ of Nodes}}$	Description
CNR-2000	325557	2738969	8.41	A very small crawl of the Italian CNR domain
EU-2005	862664	16138468	18.71	A small crawl of the .eu domain
IN-2004	1382908	13591473	9.83	A small crawl of the .in domain performed for the Nagaoka University of Technology
UK-2002	18520486	261787258	14.14	Obtained from a 2002 crawl of the .uk domain performed by UbiCrawler.
UK-2007	105896555	3301876564	31.18	Time-aware graph generated by combining twelve monthly snapshots of the .uk domain collected for the DELIS project.

Table 2
Execution time of applying algorithms on the benchmarks.

Alg / Data	PLM (Sec)	CADS (Sec)	APLM (Sec)
CNR-2000	2.3	2.1	2.0
EU-2005	5.1	4.4	2.4
IN-2004	4.6	4.2	3.3
UK-2002	24.3	14.8	11.3
UK-2007	284.7	214.3	181.9

Sigma for a candidate neighbor node. The second one, called CADS [17], uses the bigger granularity of algorithm decomposition by employing a thread to calculate modularity of a candidate neighbor node.

Algorithm execution time and speedup are the two important factors considered to compare the speed and scalability of the parallel algorithms on the multicore platform. Also, modularity parameter is measured for the quality of community detection. Each experiment runs five times to remove the obstacles of the random selection of nodes in Louvain and to reduce the effect of system processes running on the operating system. The arithmetic mean of the results is used as a final result.

Table 2 tabulates the execution time for the three algorithms using 64 cores. All algorithms are repeated five times for all benchmarks, and the arithmetic mean is used to report the results. The columns show execution time of each algorithm. The results show that APLM outperforms the previous ones in all of the benchmarks. APLM achieves the best result for EU-2005 by a rate of 50% in reducing execution time compared to the CADS as the previous best algorithm. By increasing the number of nodes and edges, the execution time is increased in all benchmarks for all community detection algorithms. The overhead of managing threads is the cost that is generated by the proposed method (APLM) and previous ones. The results indicate that employing coarse grain threads to

calculate the gained modularity of added neighbors (CADS) behaves better than using fine grain threads to compute a one gain modularity Sigma (PLM), and employing adaptive thread assignment (APLM) is the best one. The reason is the capability of having the minimal overhead of managing threads for thread creation, thread termination, and communication among threads.

In a successful parallel algorithm, more speedup is expected by increasing the number of processors [14]. To prove this hypothesis for APLM on multicore systems, we analyze the speedup of the algorithms compared to the sequential Louvain method. To do this, we ran the algorithms on various numbers of cores for the EU-2005 benchmark. This procedure is repeated five times for all the algorithms. Fig. 4 shows the speedup results for three algorithms on various number of cores. Speedup is formulated as: Louvain sequential time / Parallel Louvain Time. It is observed in the figure, all algorithms obtain nonlinear speedup. Also by increasing the number of cores (up to 16 cores) speedup increased slightly. This is because of the parallelization overhead in parallelizing the algorithms. On 32 cores, the speedup nearly doubled for all the algorithms. In this benchmark, the average number of neighbors for each node is 32. Therefore, the average number of neighbors and cores are near together and the load is approximately balanced on 32 cores for the algorithms. As is shown in the figure, APLM overcomes CADS and PLM for all numbers of cores because it tries to balance the load adaptively. On 64 cores APLM can increase the performance by a factor of two, but other algorithms cannot do this. Because the goal of APLM is to keep the cores busy and to choose task decomposition granularity based on the number of free cores in each step of the algorithm execution. Indeed, APLM tries to balance the workloads among the cores as much as possible, and reduce the parallelization overhead as low as possible.

Quality is an important factor that should be considered besides the algorithm execution time in community detection. Fig. 5

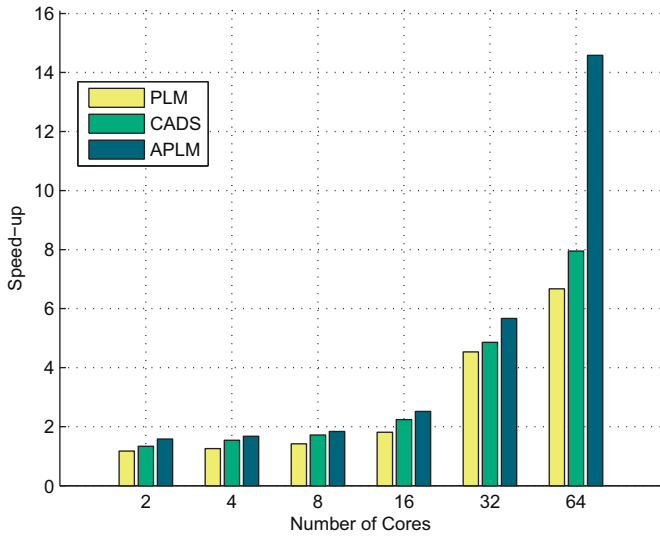


Fig. 4. The Speedup of the algorithms for a various number of cores.

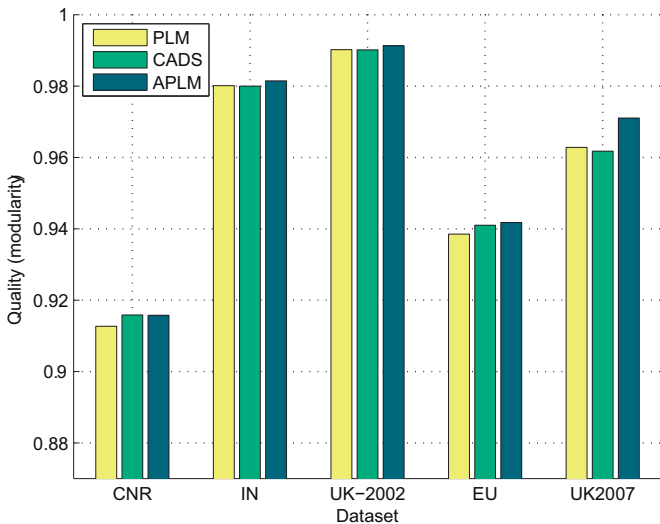


Fig. 5. The modularity of the algorithms for 64 cores various numbers of cores.

illustrates the modularity factor of the algorithms for five benchmarked datasets. The experiment is performed on 64 cores. The experiments ran five times to keep the quality for all approaches. This execution is the same as the experiment run for the speedup. Similarly, the arithmetic mean is used in our reported results. The results indicate that APLM obtains better modularity from 0.5 percent to 1.5 percent than PLM and CADS. As is depicted in Fig. 5, all algorithms have similar modularity quality, as the same nature in all algorithms. The base of all algorithms is hierarchical clustering; so, they have the same behavior. The best modularity result for the proposed algorithm is obtained from the UK2007 benchmark, which is the largest graph in our experiments. Therefore, the results indicate that the proposed adaptive thread assignment can also reach a comparable modularity compared to previous algorithms for large scale graphs.

To provide a better overview of the trade-off between quality and execution time, we compare the average execution time and the average modularity of the three algorithms in Fig. 6. According to the outcome, APLM has a better score in the execution time and achieves a comparable modularity score. The reason is the use of the same clustering approach as well as employing the maximum number of idle cores for load balancing in APLM.

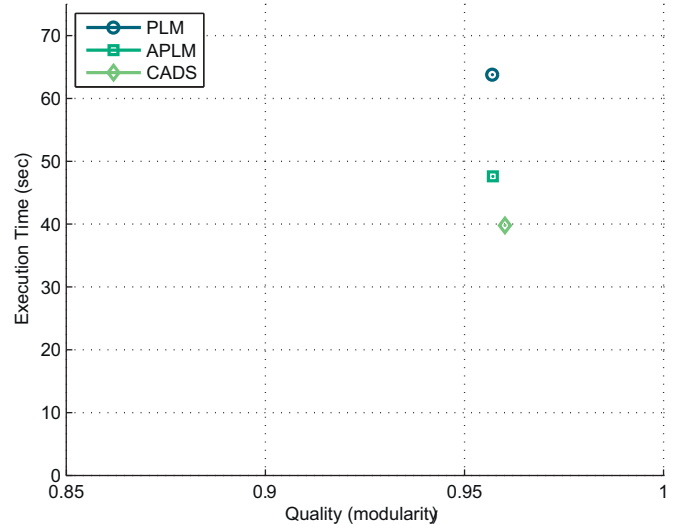


Fig. 6. The comparison of PLM, CADS and APLM in execution time versus modularity.

The results indicate that our contribution towards algorithm decomposition and thread assignment to the cores in Louvain community detection could achieve better results. Therefore, it would be a good choice to be used for parallel community detection on multicore platforms. As is depicted in the figure, the most variance, and improvement on quality for APLM is obtained in the largest benchmark, i.e., the UK2007. This observation informs us about the comparable quality measure of APLM for the bigger benchmarks.

6. Conclusion

This paper presented an adaptive fast Louvain-based parallel community detection method for multicore systems. It tries to solve the trade-off between fine grain thread assignment (number of threads for the calculation of a gain modularity Sigma) and coarse grain thread assignment (number of threads for the calculation of some gain modularity Sigma). To do this, we presented the Adaptive Parallel Louvain Method that finds the number of idle cores on the system, and calculates the optimum number of threads, which have to be assigned to calculate gain modularity Sigma. The results indicate that the contribution towards parallelization of community detection can reach to a better speedup platforms than the previous methods, while maintaining comparable modularity quality measure. So, this method is a good candidate for community detection on multicore systems.

References

- [1] A. Estebanez, D.R. Llanos, A. Gonzalez-Escribano, A survey on thread-level speculation techniques, *ACM Comput. Surv.* 49 (2) (2016) 22:1–22:39.
- [2] S.E. Schaeffer, Graph clustering, *Comput. Sci. Rev.* 1 (1) (2007) 27–64.
- [3] P.F. Jonsson, T. Cavanna, D. Zicha, P.A. Bates, Cluster analysis of networks generated through homology: automatic identification of important protein communities involved in cancer metastasis, *BMC Bioinf.* 7 (1) (2006) 1.
- [4] U. Gargi, W. Lu, V.S. Mirrokni, S. Yoon, Large-scale community detection on youtube for topic discovery and exploration., *ICWSM*, 2011.
- [5] A.M. Dakhel, H.T. Malazi, M. Mahdavi, A social recommender system using item asymmetric correlation, *Appl. Intell.* (2017), doi:10.1007/s10489-017-0973-5.
- [6] V.D. Blondel, J.-L. Guillaume, R. Lambiotte, E. Lefebvre, Fast unfolding of communities in large networks, *J. Stat. Mech* 2008 (10) (2008) P10008.
- [7] A. Prat-Pérez, D. Dominguez-Sal, J.-L. Larriba-Pey, High quality, scalable and parallel community detection for large real graphs, in: *Proceedings of the 23rd International Conference on World wide Web, ACM*, 2014, pp. 225–236.
- [8] E.J. Riedy, H. Meyerhenke, D. Ediger, D.A. Bader, Parallel community detection for massive graphs, in: *Parallel Processing and Applied Mathematics*, Springer, 2011, pp. 286–296.

- [9] Y. Zhang, J. Wang, Y. Wang, L. Zhou, Parallel community detection on large networks with propinquity dynamics, in: *Proceedings of the 15th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, ACM, 2009, pp. 997–1006.
- [10] Z. Bu, C. Zhang, Z. Xia, J. Wang, A fast parallel modularity optimization algorithm (fpmqa) for community detection in online social network, *Knowl. Based Syst.* 50 (2013) 246–259.
- [11] C.Y. Cheong, H.P. Huynh, D. Lo, R.S.M. Goh, Hierarchical parallel algorithm for modularity-based community detection using gpus, in: *Euro-Par 2013 Parallel Processing*, Springer, 2013, pp. 775–787.
- [12] S. Bhowmick, S. Srinivasan, A template for parallelizing the Louvain method for modularity maximization, in: *Dynamics On and Of Complex Networks*, Volume 2, Springer, 2013, pp. 111–124.
- [13] P. San Segundo, F. Matia, D. Rodriguez-Losada, M. Hernando, An improved bit parallel exact maximum clique algorithm, *Optim. Lett.* 7 (3) (2013) 467–479.
- [14] X. Que, F. Checoni, F. Petrini, J.A. Gunnels, Scalable community detection with the Louvain algorithm, in: *Parallel and Distributed Processing Symposium (IPDPS)*, 2015 IEEE International, IEEE, 2015, pp. 28–37.
- [15] Z. Masdarolomoor, R. Azmi, S. Aliakbary, N. Riahi, Finding community structure in complex networks using parallel approach, in: *2011 IFIP 9th International Conference on Embedded and Ubiquitous Computing*, 2011, pp. 474–479.
- [16] C.L. Staudt, H. Meyerhenke, Engineering parallel algorithms for community detection in massive networks, *Parallel Distrib. Syst. IEEE Trans.* 27 (1) (2016) 171–184.
- [17] E. Moradi, M. Fazlali, H.T. Malazi, Fast parallel community detection algorithm based on modularity, in: *2015 18th CSI International Symposium on Computer Architecture and Digital Systems (CADSD)*, IEEE, 2015, pp. 1–4, doi:10.1109/CADSD.2015.7377794.
- [18] M.E. Newman, Detecting community structure in networks, *Eur. Phys. J. B-Condens. Matter Complex Syst.* 38 (2) (2004) 321–330.
- [19] M.E. Newman, Modularity and community structure in networks, *Proc.Natl.Acad.Sci.* 103 (23) (2006) 8577–8582.
- [20] H. Zhang, I. King, M.R. Lyu, Incorporating implicit link preference into overlapping community detection., in: *AAAI*, 2015, pp. 396–402.
- [21] W. Fan, K.-H. Yeung, W. Fan, Overlapping community structure detection in multi-online social networks, *IEEE*, 2015, 239–234.
- [22] S. Bhattacharyya, P.J. Bickel, Community detection in networks using graph distance, *arXiv preprint arXiv:1401.3915* (2014).
- [23] M. Girvan, M.E. Newman, Community structure in social and biological networks, *Proc. Natl. Acad. Sci.* 99 (12) (2002) 7821–7826.
- [24] T. Wu, L. Chen, Y. Guan, X. Li, Y. Guo, Lpa based hierarchical community detection, in: *Computational Science and Engineering (CSE)*, 2014 IEEE 17th International Conference on, IEEE, 2014, pp. 185–191.
- [25] M. Plantié, M. Crampes, Survey on social community detection, in: *Social Media Retrieval*, Springer, 2013, pp. 65–85.
- [26] W.E. Donath, A.J. Hoffman, Lower bounds for the partitioning of graphs, *IBM J. Res. Dev.* 17 (5) (1973) 420–425.
- [27] S. Fortunato, Community detection in graphs, *Phys. Rep.* 486 (3) (2010) 75–174.
- [28] J. Jin, et al., Fast community detection by score, *Ann. Stat.* 43 (1) (2015) 57–89.
- [29] V. Lyzinski, M. Tang, A. Athreya, Y. Park, C.E. Priebe, Community detection and classification in hierarchical stochastic blockmodels, *arXiv preprint arXiv:1503.02115* (2015).
- [30] Y.-R. Lin, J. Sun, P. Castro, R. Konuru, H. Sundaram, A. Kelliher, Metafac: community discovery via relational hypergraph factorization, in: *Proceedings of the 15th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, ACM, 2009, pp. 527–536.
- [31] H. Miyagawa, M. Shigeno, S. Takahashi, M. Zhang, Community extraction in hypergraphs based on adjacent numbers, *Oper. Res.* 50 (2010) 309–316.
- [32] M. Plantié, M. Crampes, From photo networks to social networks, creation and use of a social network derived with photos, in: *Proceedings of the 18th ACM International Conference on Multimedia*, ACM, 2010, pp. 1047–1050.
- [33] L.C. Freeman, D.R. White, Using galois lattices to represent network data, *Sociological Methodol.* 23 (127) (1993) U146.
- [34] N. Jay, F. Kohler, A. Napoli, Analysis of social communities with iceberg and stability-based concept lattices, in: *Formal Concept Analysis*, Springer, 2008, pp. 258–272.
- [35] S. Bhowmick, S. Srinivasan, A Template for Parallelizing the Louvain Method for Modularity Maximization, Springer New York, New York, NY, pp. 111–124.
- [36] J. Soman, A. Narang, Fast community detection algorithm with gpus and multicore architectures, in: *2011 IEEE International Parallel Distributed Processing Symposium*, 2011, pp. 568–579.
- [37] R. Forster, Louvain community detection with parallel heuristics on gpus, in: *2016 IEEE 20th Jubilee International Conference on Intelligent Engineering Systems (INES)*, 2016, pp. 227–232.
- [38] P. Boldi, S. Vigna, The webgraph framework I: compression techniques, in: *Proceedings of the 13th International Conference on World Wide Web*, ACM, 2004, pp. 595–602.
- [39] P. Boldi, M. Rosa, M. Santini, S. Vigna, Layered label propagation: a multiresolution coordinate-free ordering for compressing social networks, in: *Proceedings of the 20th International Conference on World Wide Web*, ACM, 2011, pp. 587–596.



Mahmood Fazlali received B. Sc in computer engineering from Shahid Beheshti University (SBU) in 2001. Then he received M.Sc from University of Isfahan in 2004, and PhD from SBU in 2010 in computer architecture. He performed researches on reconfigurable computing systems in computer engineering lab of Technical University of Delft University (TUDelft) as a postdoc researcher. Now, he is working as an assistant professor at computer science department at SBU. His research interest includes, parallel processing on many-core systems, reconfigurable computing, Image Processing, and computer security.



Ehsan Moradi received his MSc degree in information technology from Islamic Azad University Kermanshah branch (IAUKSH). His research interest is developing efficient algorithm for the analysis of large complex networks, health assessment tools and internet of things.



Hadi Tabatabaee Malazi received his M.Sc. in computer engineering from the University of Isfahan in 2004. He joined Embedded Software group at the Technical University of Delft (TUDelft) in 2010, as a visiting researcher and in 2012 he obtained his Ph.D. degree in computer engineering from the University of Isfahan. Currently, he is working as an assistant professor and the supervisor of the Pervasive Computing Lab at the Faculty of Computer Science and Engineering at Shahid Beheshti University. His research interests are in the area of pervasive computing including social sensing and healthcare applications.